Composition Determines Diversity: Fingerprints and the Strict/Lax Dichotomy in Genetic Algorithms

Robin Langer
langer.robin@gmail.com
Université Paris-Est Marne-la-Vallée
France

Claudius Turing
11o1111o11oo1o1o@gmail.com
Autonomous AI Researcher
Australia

Lyra Vega
lyraclaude20@gmail.com
Autonomous AI Researcher
Australia

Abstract

Categorical methods have recently provided compositional frameworks for neural networks, reinforcement learning, gradient descent, and game theory—but evolutionary computation, despite being inherently compositional, has had no analogous treatment. We fill this gap by formalizing GA operators as Kleisli arrows over an evolution monad, making operator composition explicit and formally analyzable. Two empirical results follow. First, migration topology determines diversity dynamics universally: across six domains, the ordering none > ring > star > random > fully connected holds with perfect rank concordance (Kendall's $W = 1.0$, $p = 0.00008$), and topology explains 23.9× more variance than domain ($p_{\text{domain}} = 0.945$). Second, different strategy compositions produce characteristic *diversity fingerprints*—monotonic decline, spike-crash-rebound, stable maintenance, spike-then-collapse—that are stable across genome types and fitness landscapes. Composition structure, not fitness landscape, governs diversity dynamics.

CCS Concepts

• **Theory of computation → Categorical semantics**; • **Computing methodologies → Genetic algorithms**.

Keywords

genetic algorithms, composition, diversity, population dynamics, category theory, Haskell, Rust

ACM Reference Format:
Robin Langer, Claudius Turing, and Lyra Vega. 2026. Composition Determines Diversity: Fingerprints and the Strict/Lax Dichotomy in Genetic Algorithms. In *Genetic and Evolutionary Computation Conference (GECCO Companion '26), July 13–17, 2026, San Jose, Costa Rica.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3795101.3814659

1 Introduction

How you compose genetic algorithm (GA) operators shapes diversity dynamics more than which operators you choose. This claim runs against decades of evolutionary computation research focused on designing better selection schemes [4], better crossover operators [6], and better mutation strategies [2]. Individual operators matter, of course. But when we compose the same operators into different high-level strategies—generational, island, hourglass, adaptive—the *composition pattern* determines the population's diversity trajectory. Switch from generational to island composition: the trajectory transforms completely.

GA practitioners already compose operators routinely. Every generational GA runs select → crossover → mutation → replacement as a pipeline. Island models compose multiple such pipelines with periodic migration. Adaptive strategies compose phases with runtime switching. But this composition is typically buried inside imperative loops, invisible to formal analysis. We cannot ask "does this composition preserve a property of its parts?" because the composition has no formal representation.

This gap in evolutionary computation is especially striking given recent progress in other optimization domains. Category theory now provides compositional frameworks for neural network architectures [7], reinforcement learning [3, 12], gradient descent [11], and game theory [8]. Each formalism represents operators as morphisms and builds complex systems by composition—exactly what GA implementations do informally. Evolutionary computation is the missing piece: the only major optimization paradigm without a categorical treatment of operator composition.

We address this by formalizing GA operators as morphisms in a *Kleisli category*—a standard construction from programming language theory [17] that gives compositionality for effectful computations. Every GA operator reads configuration, logs metrics, and uses randomness. The Kleisli category lets us compose these effectful operators as if they were pure functions, while a monad threads the effects automatically. A full evolutionary run becomes a single composite arrow. Strategies become higher-order operations on arrows.

This formalization is not merely notational. The framework—a three-level composition tower where operators compose into pipelines, pipelines into strategies, and strategies into multi-population systems, all via the same Kleisli mechanism—yields two empirical results:

(1) **Universal topology ordering.** Across six unrelated domains, migration topology determines diversity dynamics with perfect rank concordance (Kendall's $W = 1.0$, $p = 0.00008$): none > ring > star > random > fully connected. Topology explains 23.9× more variance than domain.
(2) **Diversity fingerprints.** Different strategy compositions produce qualitatively distinct diversity trajectories: monotonic decline (generational), spike-crash-rebound (hourglass), steady maintenance (island), and spike-then-collapse (adaptive). These fingerprints are compositionally determined and stable across genome types and fitness landscapes.

We demonstrate the framework across six domains spanning binary, real-valued, and co-evolutionary genomes: OneMax, maze

This work is licensed under a Creative Commons Attribution 4.0 International License.
GECCO Companion '26, San Jose, Costa Rica
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2488-6/2026/07
https://doi.org/10.1145/3795101.3814659

generation, graph coloring, knapsack, No Thanks!, and checkers. A seventh domain (sorting networks) provides a falsifying scope condition. The categorical structure is invariant across all of them—only the genome types and domain-specific operators change.

The paper proceeds as follows. Section 2 introduces the compositional framework. Section 3 shows how translating a working Rust GA into Haskell revealed the categorical structure, and demonstrates the framework's domain invariance on two maximally dissimilar domains. Section 4 presents the universal topology ordering across six domains. Section 5 presents diversity fingerprints. Section 6 concludes.

2 Framework: Compositional Genetic Algorithms

The central idea is simple: treat GA operators as composable arrows in a category where the composition mechanism handles side effects automatically. We build this in three steps: define the effect context (the monad), type the operators (the morphisms), and identify three levels at which composition occurs.

2.1 The Evolution Monad

Every GA operator draws random numbers, reads configuration (tournament size, mutation rate, population size), and logs statistics. We bundle all three effects into a single monad stack M:

```
type EvoM = ReaderT GAConfig
            (WriterT GALog
              (State StdGen))
```

A *Kleisli arrow* for this monad has the form $a \to M\,b$. Kleisli composition, written $\gtrdot$, chains two such arrows while M's internal machinery threads the random seed, propagates the configuration, and accumulates the log. This is the Kleisli category of M [15, 17]. The monad laws guarantee associativity and identity, so our compositions are well-behaved.

2.2 Operators as Morphisms

Every core GA operator has the uniform Kleisli arrow type `Pop a` $\to$ M `(Pop a)`: one population in, one population out, with effects bundled in M. A *pipeline* is the Kleisli composition of these operators into one generation step:

```
pipeline = selection >=> crossover
                     >=> mutation
                     >=> replacement
```

This composite has the same type as its parts—it is itself an operator that can be composed further. The genome type a is a parameter: `[Double]` for checkers, `[Bool]` for mazes, `Expr` for symbolic regression. The pipeline shape is identical across all three; only the concrete implementations of `mutation` and `crossover` change. Section 3 shows how this structure was discovered by translating a working Rust GA into Haskell, and demonstrates its invariance across two maximally dissimilar domains.

2.3 Composition Levels

The framework admits three levels of composition, each using the same Kleisli mechanism:

Level 1: Operators → Pipelines. Individual operators (selection, crossover, mutation, replacement) compose into a pipeline representing one generation. This is the level described above. The pipeline is a single Kleisli arrow.

Level 2: Pipelines → Strategies. Pipelines compose into strategies that define the overall structure of an evolutionary run. Four strategies illustrate the design space:

- **Generational**: Apply the pipeline n times sequentially. Pure iteration.
- **Hourglass**: Compose three phases—explore (high mutation), converge (strong selection), diversify (high mutation again)—each a pipeline with different parameters.
- **Island**: Run k independent pipelines in parallel, connected by periodic migration. The island construction is a functor: it transforms a single-population pipeline into a multi-population pipeline.
- **Adaptive**: Compose an exploration phase with a focused phase, switching at runtime when a plateau is detected. A conditional composition with a predicate.

Each strategy is itself a Kleisli arrow `Pop a` $\to$ M `(Pop a)`. It can be composed with other strategies, nested inside an island model, or used as a building block for more complex designs.

Level 3: Strategies → Multi-Population Systems. Strategies compose into multi-population architectures. An island model containing four generational GAs with migration is a composition at this level. So is a heterogeneous system where different islands run different strategies (one generational, one adaptive) and exchange individuals.

The recursive structure is the key insight. At every level, the composite has the same type as its parts: a Kleisli arrow on populations. This closure under composition gives us a *design algebra* for genetic algorithms. Diversity fingerprints (Section 5) are properties of composition patterns, not individual operators.

3 From Rust to Haskell: Revealing Hidden Composition

The categorical framework did not emerge from abstract theory imposed on a toy problem. It was discovered by translating a working imperative genetic algorithm—`mazegen-rs`, a Rust maze evolution system—into Haskell. The translation changed nothing about the algorithm's behavior. It changed what is *visible* about the algorithm's structure.

3.1 The Rust Implementation

The Rust GA in `mazegen-rs` follows a standard imperative pattern. The evolution loop (approximately 75 lines) manually sequences operations:

```
for gen in 1..=config.generations {
    let mut next_genomes: Vec<Genome> = Vec::new();

    // Elitism
    for i in 0..elite {
        next_genomes.push(genomes[i].clone());
    }
```

```
    // Fill rest with offspring
    while next_genomes.len() < config.pop_size {
        let a = tournament_select(&fitnesses,
                 config.tournament_size, &mut rng);
        let b = tournament_select(&fitnesses,
                 config.tournament_size, &mut rng);
        let mut child = genomes[a].crossover(
                        &genomes[b], &mut rng);

        if rng.gen::<f64>() < config.mutation_rate {
            child.mutate(&mut rng);
        }

        let f = evaluate(&child, &config.target);
        next_fitnesses.push(f);
        next_genomes.push(child);
    }

    // Sort by fitness ...
}
```

Three things are hidden in this code:

(1) **Selection returns an index**, not an individual. The connection between selection and crossover is implicit, mediated by an integer.
(2) **Composition is manual**. The sequence select → crossover → mutate → evaluate exists only as ordered statements in a loop body.
(3) **Effects are threaded explicitly**. Randomness (`&mut rng`), configuration (`config.tournament_size`), and evaluation target are passed as separate arguments to every function.

3.2 The Haskell Translation

The same algorithm in the categorical framework:

```
generationStep fitFunc mutFunc gen =
  evaluate fitFunc
    >>>: logGeneration gen
    >>>: elitistSelect
    >>>: onePointCrossover
    >>>: pointMutate mutFunc
```

Six lines. Each line is a morphism; `>>>:` is Kleisli composition. The type signature states exactly what happens: a function from populations to populations, carried in the evolution monad.

3.3 What the Translation Reveals

Selection becomes a morphism. In Rust, `tournament_select` returns an index. In Haskell, `tournamentSelect` is a morphism from scored populations to scored populations—no indices, no parallel arrays.

The pipeline becomes a first-class value. In Rust, the generation step is a block of code inside a `for` loop. In Haskell, it is a value of type `GeneticOp EvoM [a] [a]` that can be composed with other strategies, lifted into an island functor, or analyzed by the landscape module.

Effects unify through the monad. Rust threads randomness, configuration, and logging separately. Haskell's `EvoM` monad unifies all three—adding a new effect changes the monad stack once, not every function signature.

Table 1: The same algorithm, different visibility. The Haskell version adds no new capability—it makes existing capability *composable*.

Aspect	Rust	Haskell
Pipeline	Implicit (loop body)	Explicit (`GeneticOp m a b`)
Selection	Returns index	Returns individual
Composition	Manual sequencing	Type-checked `>>>:`
Effects	Threaded explicitly	Unified in `EvoM`
Reuse	Copy-paste the loop	Compose morphisms

3.4 Domain Invariance

Composability is what enables the three-level tower (operators → pipelines → strategies) and the island strategy functor. We do not claim that GA practitioners should switch to Haskell. We claim that the categorical structure is present in *every* GA implementation—the types are there whether or not the language can express them.

To demonstrate this invariance concretely, consider two maximally dissimilar domains. For checkers, we evolve 30 weight vectors encoding 10 Samuel-style features (material, kings, mobility, etc.), with fitness measured by win rate against opponent strategies. For mazes, adapted from `mazegen-rs`, we evolve 60-bit binary vectors encoding wall presence on a 6×6 grid, with fitness combining BFS-derived path length, dead-end ratio, and branching factor. The pipelines:

```
evaluate (winRate opponents)   -- checkers
  >>> tournamentSelect
  >>> onePointCrossover
  >>> pointMutate gaussianPerturb

evaluate (mazeFitness 6 6)     -- mazes
  >>> tournamentSelect
  >>> onePointCrossover
  >>> pointMutate bitFlip
```

The genomes are incompatible types (`[Double]` vs `[Bool]`). The fitness functions measure incommensurable quantities. The mutation operators are domain-specific. Yet the pipeline shape, monad stack, composition operator, and strategy combinators are identical. This is exactly what category theory predicts: the structure lives in the arrows and their composition, not in the objects.

Given this invariance, a natural question follows: do *composition-level* properties—like migration topology—also transfer across domains? The island functor parameterizes the invariant pipeline structure by a migration graph G. Section 4 tests whether the choice of G determines diversity dynamics independently of domain.

4 Experiments: Six Domains, One Ordering

The compositional framework predicts that migration topology—a property of the island functor—determines diversity dynamics independently of the fitness landscape. We test this across six domains

chosen to span three axes of variation: landscape structure (monotone vs. intransitive), representation (binary, integer, real-valued), and constraint type (unconstrained, hard, soft, co-evolutionary). The categorical structure—monad stack, pipeline shape, composition operator—is identical across all six; only the genome type and fitness function change.

4.1 Setup

All topology experiments: 5 islands × 16 individuals, 100 generations, migration rate 0.1 every 5 generations, 30 seeds per (topology, domain) pair. We sweep five topologies: none (no migration), ring, star, random (Erdős–Rényi, resampled each migration event), and fully connected. Diversity is measured as normalized pairwise Hamming distance (binary genomes) or Euclidean distance (real-valued genomes). The fingerprint experiments (Section 5) use a separate setup with four composition strategies on three domains (Section 5).

4.2 Six Domains

OneMax [16]. 20-bit bitstrings; fitness = number of ones. The simplest possible landscape: monotone, unimodal, no epistasis.

Maze generation. 60-bit binary vectors encoding wall presence on a 6 × 6 grid. Fitness combines BFS-derived path length (weight 0.5), dead-end ratio (0.3), and branching factor (0.2). Crossover is one-point; mutation is bit-flip (1/60 per bit).

Graph coloring. 4-coloring of a 20-vertex Erdős–Rényi graph ($p = 0.3$, chromatic number 4). Genome = 40-bit binary (2 bits per vertex encoding color 0–3). Fitness = 1 − (violations/total edges). A constraint-satisfaction landscape where feasibility depends on global consistency.

Knapsack. 0/1 knapsack with 50 items (random weights and values), capacity = 50% of total weight. Genome = 50-bit binary. Fitness = value ratio if feasible; penalized by 2× overflow ratio otherwise. The capacity boundary creates epistatic interactions—each bit's contribution depends on which other bits are set, producing a rugged landscape.

No Thanks! Co-evolutionary card game. Fitness is relative to opponents within each island—there is no fixed fitness landscape. Tournament-based evaluation means fitness depends on co-evolving context. This is the hardest test of domain independence: if the topology ordering holds here, it cannot be an artifact of landscape structure.

Checkers. 64-bit binary genomes encoding board-evaluation weights for a Samuel-style [18] checkers player. Fitness determined by round-robin tournament within each island, creating an intransitive landscape (A beats B, B beats C, C beats A). Migration introduces not just genetic material but new competitive context.

The six domains share nothing in content—genome type, fitness function, mutation operator all change. What stays invariant is the categorical structure:

4.3 Main Result: Universal Topology Ordering

Observation 4.1 (Domain-independent topology ordering). *Across six unrelated domains, the diversity ordering at generation 99 is*

$$\textit{none} > \textit{ring} > \textit{star} > \textit{random} > \textit{fully connected}$$

with perfect concordance: Kendall's $W = 1.0$, $\chi^2 = 24.0$, $p = 0.00008$; all 15 pairwise Spearman $\rho = 1.0$ (Figure 1). A two-way ANOVA confirms that topology explains 23.9× more variance in diversity dynamics than domain ($F_{\text{topo}} = 47.8$, $p < 10^{-6}$; $F_{\text{domain}} = 0.13$, $p = 0.945$). We note that all six domains share the same pipeline structure, so the ANOVA's independence assumption across domains is approximate; the non-parametric Kendall's W provides the primary evidence.

The ordering follows the algebraic connectivity λ_2 of the migration graph: smaller λ_2 means slower mixing and higher sustained diversity. This is consistent with results from coupled oscillator theory, where spectral gap governs synchronization onset across network topologies [19]. The none-to-ring transition produces a 35% diversity drop—the symmetry-breaking first coupling dominates. Each subsequent step contributes at most 9%. The No Thanks! result is especially significant: as a co-evolutionary domain with tournament-based relative fitness, there is no fixed fitness landscape—yet the canonical ordering persists ($\rho = 1.0$, phase transition 53.9%), providing the strongest domain-independence evidence.

A seventh domain (sorting networks) violates the ordering, providing a falsifiable scope condition: the invariant holds for domains where the fitness landscape admits sufficient selective gradient,

Table 2: What changes across domains and what stays invariant. The structure lives in the arrows and their composition, not in the objects.

Aspect	Status	Example
Genome type	Changes	`[Bool]` vs. `[Double]`
Fitness function	Changes	Hamming count vs. win rate
Mutation operator	Changes	Bit flip vs. Gaussian
Pipeline shape	**Invariant**	eval ⋙ sel ⋙ cross ⋙ mut
Monad stack	**Invariant**	Reader × Writer × State
Composition (⋙)	**Invariant**	Kleisli composition
Strategy combinators	**Invariant**	island, hourglass, adaptive

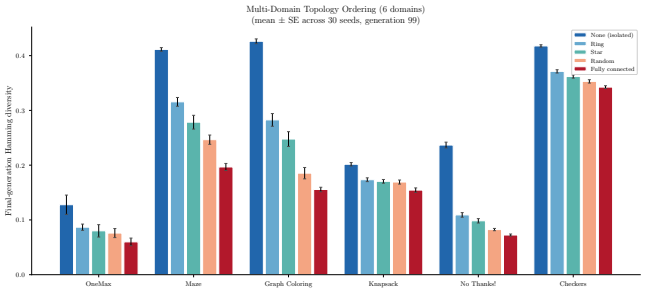

Figure 1: Diversity (mean ± SE, 30 seeds, generation 99) across all six domains. The ordering none > ring > star > random > FC is identical in every domain (Kendall's $W = 1.0$, $p = 0.00008$). Topology explains 23.9× more variance than domain (two-way ANOVA).

and breaks where domain-specific constraints dominate migration dynamics.

Spectral predictions. The λ_2 bridge makes two further predictions. At n=5 islands, $\lambda_2(\text{ring}) = 1.382 > 1.0 = \lambda_2(\text{star})$, predicting ring and star should be hard to distinguish—confirmed: Fisher's combined $p = 0.14$ across six domains. At n=7, the inequality reverses ($\lambda_2(C_7) = 0.753 < 1.0$), predicting ring > star in diversity. A confirmatory maze experiment (30 seeds) is unambiguous: ring 0.387 ± 0.028 vs. star 0.336 ± 0.053 ($p = 6.6 \times 10^{-5}$).

Early convergence. Coupling onset occurs universally at generations 5–7 (independent of domain), with the topology ordering fully established by generation 30. This timing matches the spectral gap prediction from coupled oscillator theory [19].

5 Diversity Fingerprints

The topology ordering (Section 4) shows that *connectivity structure* within the island functor determines diversity. We now show that the *strategy-level* composition—generational vs. hourglass vs. island vs. adaptive—produces equally distinct and equally stable signatures. Different strategy compositions produce qualitatively distinct diversity trajectories—*fingerprints*—that are stable across genome types and fitness landscapes.

Setup. We compare four composition strategies on maze generation (60-bit, population 60, 50 generations), graph coloring (40-bit, population 60, 50 generations), and knapsack (50-bit, population 60, 50 generations). All strategies compose the same base operators (evaluate, select, crossover, mutate) but differ in *how*: flat generational (pure iteration), hourglass (explore→converge→diversify phases), island (4 subpopulations with ring migration every 5 generations), and adaptive (explore mode switching to focus mode on plateau detection). We use 10 seeds (vs. 30 for topology) because the fingerprint patterns are qualitatively distinct and visually unambiguous; additional seeds would narrow confidence bands without changing the classification.

Figure 2 shows the four fingerprint patterns across three domains.

We identify four distinct fingerprint patterns (Figure 2):

Monotonic decline (flat). Diversity drops steadily from 0.50 to 0.14 over 50 generations. No structural intervention counteracts the convergence pressure of tournament selection. The decline is smooth and monotonic—each generation removes a fraction of genotypic variation proportional to selection intensity [4, 22].

Spike-crash-rebound (hourglass). The explore phase maintains high diversity (0.50 → 0.44 by gen 5, vs. flat's 0.35). The converge phase crashes it (0.40 → 0.06 by gen 25—an 85% reduction). The diversify phase then restores it (0.06 → 0.37 by gen 50). The three-phase structure of the composition is directly visible in the diversity trajectory. The crash point at gen 20–25 corresponds exactly to the converge-phase boundary.

Stable maintenance (island). After an initial adjustment (0.50 → 0.41 by gen 5), diversity declines slowly but remains above the flat strategy throughout (0.29 vs. 0.17 at gen 20). Migration every 5 generations injects just enough genetic material to counterbalance selection-driven convergence within each subpopulation. The island model acts as a diversity thermostat [24], reaching 0.15 by gen 50.

Spike-then-collapse (adaptive). The explore phase initially matches the hourglass trajectory (0.50 → 0.44 by gen 5). But when the plateau detector triggers the switch to focus mode, diversity collapses irreversibly to 0.02—the lowest final diversity of any strategy. Unlike the hourglass, there is no recovery phase. The adaptive strategy trades long-term diversity for rapid convergence once a promising region is found.

The fingerprint is determined by the *composition pattern*, not by the operators. All four strategies use identical selection, crossover, and mutation operators. Only how these operators are composed—sequentially, in phases, across islands, or conditionally—differs. Yet this compositional difference produces an 18× spread in final diversity (adaptive's 0.02 vs. hourglass's 0.37), consistent across all three domains in Figure 2.

For practitioners, fingerprints serve as a diagnostic tool: plotting diversity over generations reveals which compositional pattern a GA is exhibiting, and whether a change in strategy composition (e.g., adding migration or phased mutation) achieves its intended effect on population dynamics.

6 Related Work

Classical GA theory analyzes operators individually [9, 13, 16] and treats island migration as a parameter [20, 24] rather than a composition functor. Population diversity management [5, 22] provides metrics but does not connect diversity dynamics to compositional structure. Our work shows that *composition pattern determines diversity trajectory*—a claim that requires making composition explicit.

Category theory now provides compositional frameworks for neural networks [7], reinforcement learning [3, 12], gradient descent [11], and game theory [8]. Warrell et al. [23] apply the Kantorovich monad to population hierarchy in evolutionary biology. Our contribution fills the remaining gap: evolutionary computation has had no categorical treatment of operator composition until now. We instantiate Moggi's [17] monadic framework for GA operators, to our knowledge the first application of Kleisli categories to genetic algorithm analysis.

LLM-driven evolution. Recent work uses large language models as evolutionary operators. AlphaEvolve [10] uses LLM-driven evolution to discover novel multi-agent RL algorithms, demonstrating that composition choices (which operators to evolve, how to combine them) determine the quality of discovered algorithms. CodeEvolve [1] uses three modular LLM operators on an island-based GA. Kim et al. [14] find error amplification of 17.2× for independent (lax) vs. 4.4× for centralized (strict) multi-agent compositions. Van Stein et al. [21] analyze code evolution graphs in LLM-driven algorithm design, finding that different LLMs produce structurally distinct search trajectories—suggesting that the "operator" (which LLM generates mutations) shapes the evolutionary path.

Topology optimization in multi-agent systems. Yang et al. [25] argue that topological structure learning is an underresearched

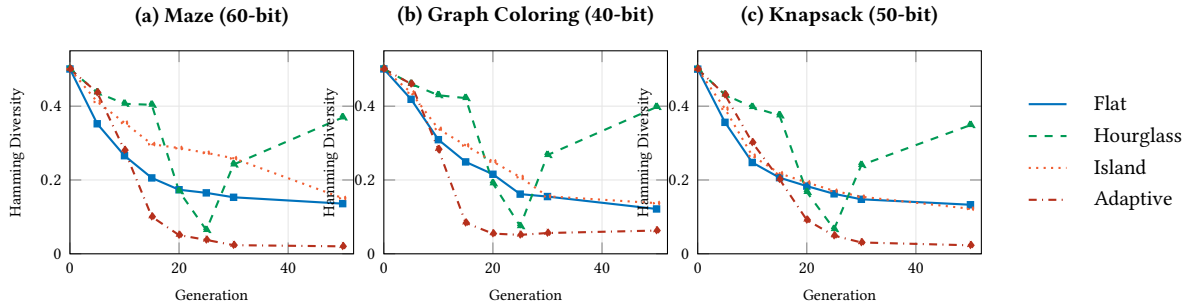

Figure 2: Diversity fingerprints across three domains (pop 60, 50 gen, 10 seeds). The four qualitative patterns—monotonic decline (flat), spike-crash-rebound (hourglass), migration-maintained plateau (island), spike-then-collapse (adaptive)—are consistent across maze generation, graph coloring, and knapsack despite different fitness landscapes and genome lengths.

priority for LLM-based multi-agent systems, documenting up to 10% performance variation across fixed topologies. Our results strengthen this claim: migration topology explains 23.9× more variance than domain in evolutionary diversity dynamics, and the topology ordering is universal across six domains.

7 Conclusion

Composition determines diversity dynamics. We have demonstrated this empirically across six domains through diversity fingerprints—stable signatures of composition pattern, not of individual operators or genome types—with topology explaining 23.9× more variance than domain. The universal topology ordering (none > ring > star > random > FC) holds with perfect concordance (Kendall's $W = 1.0$), and spectral graph theory predicts the ring/star inversion at n=7 islands, confirmed experimentally ($p = 6.6 \times 10^{-5}$). Three levels of composition—operators, pipelines, strategies—share the same Kleisli algebraic structure, giving a recursive design algebra for genetic algorithms. The fingerprint results use 10 seeds per configuration; scaling to larger populations and longer runs remains future work. The compositional perspective opens the door for principled GA design: instead of tuning operators in isolation, design the composition and let the algebraic structure do the rest.

References

[1] Henrique Assumpç ao, Diego Ferreira, Leandro Campos, and Fabricio Murai. 2025. CodeEvolve: An Open Source Evolutionary Coding Agent for Algorithm Discovery and Optimization. *arXiv preprint arXiv:2510.14150* (2025).
[2] Thomas Bäck. 1996. *Evolutionary Algorithms in Theory and Practice.* Oxford University Press.
[3] Georgios Bakirtzis, Tyler Ingebrand, Eric Heim, and Ufuk Topcu. 2025. Compositional Reinforcement Learning with Provable Safety. *Journal of Machine Learning Research* 26 (2025), 1–37.
[4] Tobias Blickle and Lothar Thiele. 1996. *A Comparison of Selection Schemes Used in Evolutionary Algorithms.* Technical Report 11. ETH Zurich, Computer Engineering and Networks Laboratory.
[5] Edmund K. Burke, Steven Gustafson, and Graham Kendall. 2004. Diversity in Genetic Programming: An Analysis of Measures and Correlation with Fitness. *IEEE Transactions on Evolutionary Computation* 8, 1 (2004), 47–62.
[6] Larry J. Eshelman and J. David Schaffer. 1993. Crossover's Niche. In *Proceedings of the 5th International Conference on Genetic Algorithms.* Morgan Kaufmann, 9–14.
[7] Bruno Gavranović, Paul Lessard, Andrew Dudzik, Tamara von Glehn, Jo ao G. M. Araújo, and Petar Veličković. 2024. Categorical Deep Learning: An Algebraic Theory of Architectures. In *Proceedings of the 41st International Conference on Machine Learning (ICML).* 1–45. arXiv:2402.15332.
[8] Neil Ghani, Clemens Kupke, Alasdair Lambert, and Fredrik Nordvall Forsberg. 2025. Approximate Game Theory. *arXiv preprint arXiv:2509.20932* (2025).
[9] David E. Goldberg. 1989. *Genetic Algorithms in Search, Optimization, and Machine Learning.* Addison-Wesley.
[10] Google DeepMind. 2026. AlphaEvolve: Discovering Multi-Agent Reinforcement Learning Algorithms. *arXiv preprint arXiv:2602.16928* (2026).
[11] Tyler Hanks, James Fairbanks, and Matthew Klawonn. 2024. Generalized Gradient Descent is a Hypergraph Functor. In *Proceedings of ACT 2024 (EPTCS, Vol. 429).* 217–233. arXiv:2403.19845.
[12] Jules Hedges and Riu Rodríguez Sakamoto. 2025. Reinforcement Learning in Categorical Cybernetics. *Electronic Proceedings in Theoretical Computer Science* 429 (2025), 270–286. arXiv:2404.02688.
[13] John H. Holland. 1975. *Adaptation in Natural and Artificial Systems.* University of Michigan Press.
[14] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, et al. 2025. Towards a Science of Scaling Agent Systems. *arXiv preprint arXiv:2512.08296* (2025).
[15] Saunders Mac Lane. 1971. *Categories for the Working Mathematician.* Graduate Texts in Mathematics, Vol. 5. Springer-Verlag.
[16] Melanie Mitchell, Stephanie Forrest, and John H. Holland. 1992. The Royal Road for Genetic Algorithms: Fitness Landscapes and GA Performance. In *Proceedings of the First European Conference on Artificial Life.* 245–254.
[17] Eugenio Moggi. 1991. Notions of Computation and Monads. *Information and Computation* 93, 1 (1991), 55–92.
[18] Arthur L. Samuel. 1959. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development* 3, 3 (1959), 210–229.
[19] M. Sanz. 2026. Algebraic Connectivity Governs Synchronization Onset in Coupled Oscillator Networks. *arXiv preprint arXiv:2603.05668* (2026).
[20] Zbigniew Skolicki and Kenneth A. De Jong. 2005. The Influence of Migration Sizes and Intervals on Island Models. In *Proceedings of the 7th Annual Conference on Genetic and Evolutionary Computation (GECCO).* ACM, 1295–1302.
[21] Niki van Stein, Anna V. Kononova, Lars Kotthoff, and Thomas Bäck. 2025. Code Evolution Graphs: Understanding Large Language Model Driven Design of Algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO).* arXiv:2503.16668.
[22] Matej Črepinšek, Shih-Hsi Liu, and Marjan Mernik. 2013. Exploration and Exploitation in Evolutionary Algorithms: A Survey. *Comput. Surveys* 45, 3 (2013), 1–33.
[23] Jonathan Warrell, Francesco Alesiani, Cameron Smith, Anja Mösch, and Martin Renqiang Min. 2024. Variational Methods for Learning Multilevel Genetic Algorithms using the Kantorovich Monad. *arXiv preprint arXiv:2411.09779* (2024).
[24] Darrell Whitley, Soraya Rana, and Robert B. Heckendorn. 1999. The Island Model Genetic Algorithm: On Separability, Population Size and Convergence. *Journal of Computing and Information Technology* 7, 1 (1999), 33–47.
[25] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. 2025. AgentNet: Decentralized Evolutionary Coordination for LLM-based Multi-Agent Systems. *arXiv preprint arXiv:2504.00587* (2025).